# Demo script: Case Study Engine (about 9 minutes + Q&A)

Run every command from the `submission` folder. Every command below was tested. Each demo is tagged with the judging criterion it scores on: **[Agentic]**, **[Business]**, **[Buyer]**, **[Process]**.

---

## Part A. Setup (10 minutes before the mentors arrive)

1. Open a terminal in the folder: `bash cd C:\Users\Mohit\Downloads\psi\submission`
2. Reset to a clean state (no approvals, no bad-draft project): `bash python demo/reset_demo.py`
3. Warm-up build (creates every output and the viewer): `bash python engine/run.py build --project drishti`
4. Open the review screen in Chrome or Edge. It's one self-contained file: `bash start runs\drishti\viewer.html`
5. Keep these ready in other windows:
   - `PITCH.pptx` on slide 1;
   - `PROCESS.md`;
   - the original snapshot `..\Project_Drishti_Library\Project_Drishti_Library\10_media\cam_PK2-07_alert_snapshot_2026-06-29.jpg`.
6. Make the terminal font large and clear the screen.

**Check before you start:** the viewer header shows `verifier: 0 problems`, `internal: stopped`, `client: stopped`, and the Overview tab is open.

---

## Part B. The demo

### Demo 0: The problem (30 s): Pitch slide 1

> "Vendors win enterprise deals with proof, but clients sign NDAs, so the best stories sit in email threads. Someone spends days working out what was really promised, which numbers are true, what actually caused the results, and what the contract allows. My engine does that in minutes, and nothing leaves without the right people approving."

---

### Demo 1: One command, a chain of agents (1 min) [Agentic]

```
python engine/run.py build --project drishti
```

Point at each log line:

> "The Librarian indexes 42 files line by line. The Rules agent reads the **contract first**: what may be said, who must approve, and which words are banned. The Results agent picks the KPI definitions from the signed requirements. The Calculator is plain code: it computes all 44 numbers from the raw CSVs, so no number comes from the language model. Exposure scans for anything identifying, including GPS hidden inside image files. The Writers produce the outputs, and Media draws the charts. The Verifier checks every line: zero problems. The Gate assembles the pack, hashes it, and **stops**."

Say this before anyone asks:

> "The log says RECORDED for the LLM agents. There was no API key on this laptop, so those agents' outputs were written with the exact prompts in `engine/prompts/`, and the engine replays them. With a Gemini 3 key in `.env` the same calls go live, including a fixer agent that gets refused lines back from the verifier. The code agents run live every time."

---

### Demo 2: The dashboard (1 min) [Agentic] [Business]

Refresh the viewer and stay on **Overview**.

1. Headline cards:

   > "Baseline to result, against the target the client signed. Two targets were met and two were missed in the pilot, and the dashboard says so. The cards also show what else contributed."

2. Agent chain:

   > "Every agent, its job, whether it's code or LLM, and what it did in this run."

3. Approval gate:

   > "The verifier passed. Internal and client approval are both stopped, waiting for a person."

4. Judging criteria panel:

   > "And here is where each judging criterion's evidence is." Click **Results sheet** under Business understanding.

---

### Demo 3: Every number traces to a source (1.5 min) [Business]

On the **Results sheet** tab:

1. Hover the green chip `C:k1_final`:

> "26,081 compliant out of 28,634 observations is 91.1%. It matches the client's signed acceptance certificate to the decimal for every pilot KPI."

2. Hover a blue chip:

> "This one shows the exact file and line."

3. **Where sources disagree**:

> "The walk score says 74%, the camera baseline 69%, a vendor email 96.6%. The engine picks the signed definition and says why."

4. **Numbers we refuse to use**:

> "The sales headline: '74 to 97%, zero LTIs, ₹2.3 crore'. It mixes baselines, uses one zone's best week, and counts savings the client's Finance rejected. Refused."

5. Gate section:

> "Turnaround fell 24%. About 6 minutes came with the client's own new weighbridge, and 15 after their dispatch slots. We share the credit."

6. Click **Story spine** and scroll to the promised / signed / delivered table:

> "Every promise, what was signed, what was delivered, and why it changed."

---

### Demo 4: It refuses a bad claim (1 min) [Agentic]

```
python demo/make_bad_draft.py
```

```
python engine/run.py build --project drishti_bad
```

> "I slip in the sales team's real headline, the zero-LTI line, and the plant name with its camera count."

Point at **"6 untraceable/unsafe items ... REFUSED: pack not assembled"**. Then:

```
start runs\drishti_bad\viewer.html
```

Open the **Verifier** tab:

> "74 and 97 aren't what the cited lines say. The LTI sentence has no source. The plant name and 'cement major' are banned identifiers. No pack exists, so there's nothing to approve. The verifier also catches citations that point at the wrong line, and estimates that aren't marked ASSUMED."

Close that tab and go back to the drishti viewer.

---

### Demo 5: Exposure and media (1 min) [Buyer] [Business]

Click **Exposure**, then show the original snapshot file:

> "The client's own camera snapshot shows the company name on a signboard and an internal IP, and the file hides the plant's **GPS coordinates**. The sales deck wanted to use it. The engine also caught a combination risk: the client's Head of EHS posted publicly naming us, so the same story in the same words would identify them."

Click **Media pack**:

> "So everything is redrawn from data, with the metadata stripped and checked: compliance by camera cohort, gate time with the client's actions marked, a generic architecture, an illustration labelled as one, and an ROI calculator the buyer fills in with their own numbers."

Change a value in the calculator to show it working.

---

### Demo 6: One story, three readers (1 min) [Buyer]

Click **Case study** and scroll:

> "The same facts, told three ways. **Gatekeeper:** on-prem, no internet egress, text-only phone alerts, the face module removed (not just switched off) with the hash verified by their IT, and the outage handled openly. **User:** a normal day before and after. False alarms were 23% at first and are now checked weekly by their own team. The photo ended arguments, and gang leaders now check helmets themselves. **Buyer:** one headline, 69 to 91%, with money only where Finance measured it, marked ASSUMED where it's an estimate. **Champion:** five lines they can forward to their boss."

Click **Approval request**:

> "It goes to Corporate Comms and Legal by email, because the contract says plant staff alone can't approve. There's a firm reply date that respects the 10-business-day review, options per asset, every number with its source, and we offer them the charts for their own council deck. Saying yes is easy and good for them."

---

### Demo 7: It stops for a person (1.5 min) [Process]

Run one command at a time:

```
python engine/run.py release --project drishti --stage internal
```

> "STOP: nobody has approved."

```
python engine/run.py approve --project drishti --stage internal --role vendor_delivery_owner --name "Delivery Manager" --decision approve
```

```
python engine/run.py release --project drishti --stage internal
```

> "Our delivery owner checked it, so it can go to the client. It still can't be published."

```
python engine/run.py approve --project drishti --stage client --role unit_head --name "Unit Head" --decision approve --channel whatsapp
```

```
python engine/run.py release --project drishti --stage client
```

> "This is what the sales team tried in the library: a plant head's yes on WhatsApp. The contract needs Corporate Comms and Legal, so it's still stopped."

```
python engine/run.py approve --project drishti --stage client --role client_corporate_communications --name "Corporate Comms" --decision approve_anonymised --sent-on 2026-11-02 --first-use 2026-11-06
```

```
python engine/run.py approve --project drishti --stage client --role client_legal --name "Corporate Legal" --decision approve_anonymised
```

```
python engine/run.py release --project drishti --stage client
```

> "Both have approved, but publishing four days later breaks the 10-business-day review, so it's stopped."

```
python engine/run.py approve --project drishti --stage client --role client_legal --name "Corporate Legal" --decision approve_anonymised --first-use 2026-11-17
```

```
python engine/run.py release --project drishti --stage client
```

> "RELEASE OK."

```
python engine/run.py withdraw --project drishti --stage client --role client_legal --date 2027-01-10
```

```
python engine/run.py release --project drishti --stage client
```

> "If Legal changes their mind later, new use stops. Approvals are tied to the pack's hash, so any edit voids them."

Refresh the viewer: the gate cards and the **project register** on the Overview now show the history.

*(Optional, if asked "what happens on a no?")*

```
python engine/run.py approve --project drishti --stage client --role client_legal --name "Corporate Legal" --decision reject --note "wait for final audit"
```

> "It prints the 'on a no' steps, and the register marks the project declined, with the reason."

---

### Demo 8: Tests and the process (1 min) [Agentic] [Process]

```
python -m pytest -q tests
```

> "Twelve checks. They cover refusing claims, stopping for a person, a client's no, and the same code running a completely different software project with only a new metric spec."

Show `PROCESS.md`:

> "Triggered when acceptance is signed. Two human stops. On a no, we make one smaller ask and never chase. On a withdrawal, new use stops within 30 days. Every run keeps its evidence index, ledger, verifier report, approvals and hash, and the register shows every project at a glance. For 50 projects a year the code doesn't change: each project's own contract and data drive it."

Close with pitch slide 3:

> "And timing matters. The client's champion said to wait for the final audit before asking for a named version, so we ask for anonymised now and named later."

---

## Part C. Likely mentor questions

| Question | Answer |
|---|---|
| "Did an LLM actually run?" | The judgement agents' outputs were written with the prompts in `engine/prompts/` and are replayed (RECORDED), because there was no key here. With `GEMINI_API_KEY` in `.env` the same calls run live on Gemini 3, plus the fixer loop. All maths, scanning, verification and gating is code and runs live. |

| Question | Answer |
| --- | --- |
| "How do you stop hallucinated numbers?" | The LLM never computes. It writes a metric spec, and code runs it. The verifier then checks every number against the cited line or calculation, and checks the citation is relevant. |
| "What can the verifier miss?" | Full sentence meaning and date values (dates still need a citation). That's why there's a human internal approval. |
| "Why 69% and not 74%?" | 74% is the announced walk score. The signed requirements set the camera-measured first two weeks (68.9%) as the baseline. |
| "Isn't zero LTIs your best result?" | Seven months at about one LTI per quarter proves nothing, and the client's Head of EHS refused to claim it. |
| "How much money did it save?" | Only what Finance measured: detention down about two thirds in one month (a September estimate, marked ASSUMED), with credit shared. Their simple payback is "a little over two years", also ASSUMED. |
| "Why not publish before the Expo?" | The contract needs 10 business days and Corporate Comms + Legal, and the champion advised waiting for the final audit. |
| "A new project tomorrow?" | `--library <folder>`. The Rules and Results agents write that project's terms and spec; the code is unchanged (test 10). |

## Reset after the demo

```
python demo/reset_demo.py
```